Talenteo Technical Documentation

Talenteo HR System — Technical Documentation

Project Overview

Talenteo is a backend REST API built as a technical exercise to demonstrate software engineering practices in a real-world HR domain. The system covers two core business areas:

- **Employee Management** — create, update, deactivate, and search employees with filtering and pagination
- **Payroll Processing** — calculate, review, recalculate, finalize, and bulk-process monthly payroll slips with configurable tax rules

The design emphasizes clean architecture, thoughtful decision-making, and production-grade engineering practices rather than just making things work.

Tech Stack

Layer	Technology
Language	Java 21
Framework	Spring Boot 3.4.4
Database	PostgreSQL
Schema migrations	Flyway
ORM	Spring Data JPA / Hibernate
Object mapping	ModelMapper
Validation	Bean Validation (Jakarta)
API documentation	Springdoc OpenAPI (Swagger UI)
Error handling	RFC 7807 Problem Details
Testing	JUnit 5, Mockito, Testcontainers
Build	Maven

Architecture Highlights

- **Package-by-feature** — each feature (`employee` , `payroll`) is self-contained with its own controller, service, repository, domain, and DTO layers
- **Two-step payroll flow** — calculate (DRAFT) → review → recalculate if needed → finalize (FINALIZED), making finalization meaningful and immutability intentional
- **Config-driven tax rules** — rates externalized to `application.yml` via `@ConfigurationProperties` , no magic numbers in business logic
- **Salary snapshot pattern** — salary and bonus are copied into line items at calculation time, ensuring past payroll slips are never affected by future employee updates
- **Defense in depth** — business rules enforced at both service and database levels (unique constraints, check constraints)

Document Structure

Document	Content
ER Diagram	Entity-relationship diagram showing the data model
Decision Records	Architecture and use case decisions with rationale
Requirements	Functional use cases derived from the specification
Package Structure	Project layout and package-by-feature rationale
Technical Decisions	Technology choices and trade-offs
API Contract	All endpoints with request/response shapes
User Stories	35 developer stories across 5 epics, with completion status

```
erDiagram
    EMPLOYEE {
        bigint id PK
        varchar first_name "NOT NULL"
        varchar last_name "NOT NULL"
        varchar email "UNIQUE, NOT NULL"
        date hire_date "NOT NULL"
        varchar department "NOT NULL"
        varchar position "NOT NULL"
        decimal base_salary "NOT NULL, > 0"
        decimal bonus "DEFAULT 0"
        enum status "ACTIVE | INACTIVE"
        timestamp created_at
        timestamp updated_at
    }

    PAYROLL_SLIP {
        bigint id PK
        bigint employee_id FK
        smallint period_year "NOT NULL"
        tinyint period_month "NOT NULL, 1-12"
        decimal net_salary "NOT NULL, computed at calculation time"
        enum status "DRAFT | FINALIZED"
        timestamp calculated_at
    }

    PAYROLL_LINE_ITEM {
        bigint id PK
        bigint payroll_slip_id FK
        enum type "EARNING | DEDUCTION"
        enum code "BASE_SALARY | BONUS | INCOME_TAX | SOCIAL_CONTRIBUTION |
HIGH_SALARY_SURCHARGE"
        varchar label "human-readable snapshot"
        decimal amount "NOT NULL"
    }

    EMPLOYEE ||--o{ PAYROLL_SLIP : "has many"
    PAYROLL_SLIP ||--o{ PAYROLL_LINE_ITEM : "has many"
```

> ***Unique constraint:*** *`PAYROLL_SLIP(employee_id, period_year, period_month)` — enforced at DB level.*
>
> ***Net salary*** *is stored as a denormalized column on `PAYROLL_SLIP`, computed once at calculation time (`SUM(EARNING) - SUM(DEDUCTION)`) and never recalculated.*
>
> ***Immutability:*** *once `PAYROLL_SLIP.status = FINALIZED`, no modifications allowed — enforced in the service layer.*
>
> ***Department and Position*** *are plain `VARCHAR` fields on `EMPLOYEE` — no separate tables or management APIs.*

Architecture Decision Records

A living document of design decisions made during the conception phase.

DM-001 — Department and Position as inline string fields on Employee

Options:

- A: Plain `VARCHAR` fields on the employee table
- B: FK references to dedicated `departments` and `positions` tables

Decision: Option A — inline `VARCHAR` fields on the employee table. No separate tables, no dedicated APIs.

Rationale: Keeping department and position as simple string fields reduces scope and complexity without losing meaningful information. No management endpoints are needed for them — they are set inline when creating or updating an employee.

DM-002 — Employee deletion is a soft delete

Options:

- A: Hard delete — `DELETE FROM employees WHERE id = ?`
- B: Soft delete — `status` field (`ACTIVE` , `INACTIVE`), never remove the row

Decision: Option B — soft delete via `status` enum.

Rationale: HR systems must preserve history. A terminated employee's payroll records must remain queryable. Hard delete would orphan or cascade-delete payroll data.

DM-003 — Payroll modeled as Header + Line Items, not a flat table

Options:

- A: Flat table — one row per payroll with fixed columns
- B: `payroll_slip` as header + `payroll_line_item` rows for each earning/deduction

Decision: Option B — header + line items.

Rationale: Flat table is rigid — adding a new deduction type requires a schema change. Line items support variable structure, mirror real payroll systems, and make immutability natural.

DM-004 — Net salary stored as denormalized column on payroll_slip

Options:

- A: Compute at query time: `SUM(EARNING) - SUM(DEDUCTION)` — no storage needed
- B: Denormalized column `net_salary` on `payroll_slip` — computed once at calculation time and stored

Decision: Option B — stored in DB.

Rationale: Payroll is immutable once finalized, so the risk of inconsistency is eliminated. Storing net salary avoids an aggregation query on every read, and makes the value immediately available in list responses without joining line items.

DM-005 — Payroll immutability enforced at two levels

Decision:

1. **Database level** — unique constraint on `(employee_id, period_year, period_month)` prevents duplicate slips

2. **Service level** — once `status = FINALIZED`, the service rejects modifications

Rationale: Defense in depth. DB constraint is the last line of defense; service check gives a meaningful error to the API consumer.

DM-006 — Salary and bonus snapshotted as line items at calculation time

Decision: At calculation time, `base_salary` and `bonus` are copied from the employee record into `EARNING` line items. They are never referenced by FK.

Rationale: If the employee's salary changes later, past payroll slips are untouched. The `label` field also snapshots the rate (e.g., `"Income Tax (20%)"`) so rule changes don't affect historical records.

UC-001 — Payroll calculation and finalization are two separate steps

Options:

- A: Calculate and finalize in one step — payroll is immediately immutable
- B: Two steps — calculate (DRAFT), then explicitly finalize (FINALIZED)

Decision: Option B — two separate steps.

Rationale: HR needs to review the payroll slip before committing it. The DRAFT state allows verification. Only after HR validates does the slip become FINALIZED and immutable.

UC-002 — Payroll can be calculated for past and future periods

Decision: No restriction on the period date. HR can calculate payroll for any month/year combination.

Rationale: HR may need to process backdated payroll (late hires, corrections) or prepare payroll in advance.

UC-003 — Duplicate payroll returns 409

Decision: If a payroll slip already exists for a given `(employee_id, period_year, period_month)`, the API returns `409 Conflict`.

Rationale: Silent overwrite would be dangerous in a financial context. The HR must be explicit — if a recalculation is needed, the existing slip must be handled first.

UC-004 — Bulk payroll calculation is in scope

Decision: The system supports calculating payroll for all active employees for a given period in a single request.

Rationale: In practice, HR runs payroll for the entire company at once, not one employee at a time. Individual calculation remains available for corrections or new hires.

Conflict behavior: Employees who already have a slip for that period are silently skipped. The rest are processed normally. The response reports which were created and which were skipped.

DM-007 — Tax and deduction rules hardcoded in service

Decision: Rules are externalized to `application.yml` via `@ConfigurationProperties`. The calculator reads rates from an injected config object — no magic numbers in code.

Config structure (`application.yml`):

```yaml
payroll:
  rules:
    income-tax-rate: 0.20
    social-contribution-rate: 0.05
    high-salary-threshold: 5000.00
    high-salary-surcharge-rate: 0.05
```

Rules applied:

Rule	Code	Type	Calculation
Base salary	`BASE_SALARY`	EARNING	Employee's current base salary
Bonus	`BONUS`	EARNING	Employee's bonus (if > 0)
Income tax	`INCOME_TAX`	DEDUCTION	`baseSalary × income-tax-rate`
Social contribution	`SOCIAL_CONTRIBUTION`	DEDUCTION	`baseSalary × social-contribution-rate`
High salary surcharge	`HIGH_SALARY_SURCHARGE`	DEDUCTION	`baseSalary × high-salary-surcharge-rate`, only if `baseSalary > high-salary-threshold`

Rationale: Tax rates change in real systems without code changes. Externalizing rates to config means a rate update requires only a config change and restart — not a recompile and redeploy. `PayrollCalculator` remains a pure service injected with a config bean, keeping it fully testable. A DB rules table (Option C) would be more flexible but is over-engineering for this exercise scope.

UC-005 — Recalculation of DRAFT payroll is supported

Decision: A DRAFT payroll slip can be recalculated. Recalculation deletes the existing DRAFT slip and its line items, then runs a fresh calculation from the employee's current salary/bonus and current config rules. Finalized slips cannot be recalculated.

Rationale: Without recalculation, the two-step calculate → finalize flow has no purpose. Recalculation gives HR the ability to correct a slip before committing it. It also makes finalization meaningful — it signals "this version is locked, no further changes allowed."

Future extensions (out of scope for this exercise):

- **Manual line item adjustment** — allow HR to add, remove, or override individual line items on a DRAFT slip before finalizing (e.g. one-time bonus, manual correction)
- **Rule-based recalculation** — if tax rules are moved from `application.yml` to a database table (see DM-007), recalculation would pick up the latest rules automatically, enabling retroactive rule changes without a redeploy

Requirements & Use Cases

Actor: HR Manager (the only actor in scope)

Employee Management

UC-EM-01 — Create an employee

The HR manager can create a new employee by providing their personal information, employment details, and compensation data.

Preconditions: The email must not already exist in the system.

Outcome: A new employee is created with status `ACTIVE`.

UC-EM-02 — View an employee

The HR manager can retrieve the full profile of an employee by their ID.

Outcome: Returns personal info, employment details, compensation data, and current status.

UC-EM-03 — List employees

The HR manager can retrieve a list of all employees.

- By default, returns all employees including `INACTIVE` ones
- HR can filter by status (`ACTIVE` / `INACTIVE`)
- HR can filter by department (plain string match)

UC-EM-04 — Update an employee

The HR manager can update an employee's information (e.g., salary raise, department change, name correction).

Preconditions: Employee must exist.

Note: Updating salary does not affect past payroll slips — those are already snapshotted.

UC-EM-05 — Deactivate an employee

The HR manager can deactivate an employee when they leave the company.

Outcome: Employee status changes to `INACTIVE`. The employee record and all payroll history are preserved.

Payroll

UC-PR-01 — Calculate payroll for one employee

The HR manager can trigger a payroll calculation for a specific employee and a given period (month + year).

Preconditions:

- Employee must exist and be `ACTIVE`
- No payroll slip must already exist for that employee and period — otherwise `409 Conflict`

Outcome: A `DRAFT` payroll slip is created with all earning and deduction line items snapshotted.

Note: Past and future periods are allowed.

UC-PR-02 — Calculate payroll in bulk

The HR manager can trigger payroll calculation for all `ACTIVE` employees for a given period in a single request.

Preconditions: No payroll slip must already exist for any of the targeted employees for that period.

Behavior: If some employees already have a slip for that period, those are skipped — the rest are calculated normally.

Outcome: A `DRAFT` payroll slip is created for each eligible active employee.

UC-PR-03 — Finalize a payroll slip

The HR manager can finalize a `DRAFT` payroll slip after reviewing it.

Preconditions: The payroll slip must exist and be in `DRAFT` status.

Outcome: Slip status changes to `FINALIZED`. No further modifications are possible.

UC-PR-04 — View a payroll slip

The HR manager can retrieve a specific payroll slip for an employee and period.

Outcome: Returns the slip header (period, status, calculated_at) and all line items. The net salary is computed from the line items.

UC-PR-05 — List payroll history for an employee

The HR manager can retrieve all payroll slips for a given employee, ordered by period descending.

Outcome: Returns a list of slips with their period, status, and net salary.

Package Structure

Approach: Package by Feature with internal layering.

Each feature is self-contained. Inside each feature, the standard Controller/Service/Repository layering is explicit and visible.

```
com.talenteo.hr
│
├── employee
│   ├── controller
│   │   └── EmployeeController
│   ├── service
│   │   └── EmployeeService
│   ├── repository
│   │   └── EmployeeRepository
│   ├── domain
│   │   └── Employee                    ← JPA entity
│   └── dto
│       ├── EmployeeRequest
│       └── EmployeeResponse
│
├── payroll
│   ├── controller
│   │   └── PayrollController
│   ├── service
│   │   ├── PayrollService              ← orchestration: save, retrieve, finalize
│   │   └── PayrollCalculator           ← pure math: computes line items from salary/bonus
│   ├── repository
│   │   ├── PayrollSlipRepository
│   │   └── PayrollLineItemRepository
│   ├── domain
│   │   ├── PayrollSlip                 ← JPA entity (header)
│   │   └── PayrollLineItem             ← JPA entity (line items)
│   └── dto
│       ├── PayrollCalculationRequest   ← single employee calculation
│       ├── BulkPayrollRequest          ← bulk calculation (period only)
│       ├── PayrollSlipResponse         ← slip header + line items + net salary
│       └── PayrollSummaryResponse      ← used in history list (no line items)
│
└── common
    ├── exception
    │   ├── ResourceNotFoundException
    │   ├── PayrollAlreadyExistsException
    │   ├── PayrollAlreadyFinalizedException
    │   └── GlobalExceptionHandler      ← @ControllerAdvice
    └── dto
        └── ErrorResponse
```

Key Decisions

Why package by feature? Opening `payroll/` gives you everything about payroll. No jumping between `controller/`, `service/`, `repository/` folders at the root level. Adding a new feature means adding a new folder — existing features are untouched.

Why internal layering inside each feature? The evaluation explicitly checks Controller/Service/Repository separation. The internal `controller/`, `service/`, `repository/` sub-packages make this separation visible and unambiguous.

Why `PayrollService` and `PayrollCalculator` as two separate classes?

- `PayrollCalculator` — pure math, no Spring dependencies, no DB access. Takes a salary and bonus, returns a list of line items. Trivially unit testable without any mocks.
- `PayrollService` — orchestration: fetch the employee, call the calculator, enforce business rules (duplicate check, finalization guard), persist the result.

Why no `department/` or `position/` feature packages? Department and position are inline `VARCHAR` fields on the Employee entity. They have no dedicated APIs, no separate tables, and no management lifecycle. They live entirely within the `employee` feature.

What belongs in `common/`? Only two things:

- Exceptions — because `GlobalExceptionHandler` must see all of them
- `ErrorResponse` DTO — the unified error envelope returned by the handler

Nothing else goes in `common/`. It is not a utility dumping ground.

Technical Decisions

Cross-cutting technical decisions for error handling and validation.

TD-001 — Error handling follows RFC 7807 (Problem Details)

Decision: All error responses follow RFC 7807 using Spring Boot's native `ProblemDetail` support. No external library (e.g. Zalando) needed.

Standard fields:

- `type` — URI identifying the problem type (defaults to `about:blank`)
- `title` — short human-readable summary of the problem
- `status` — HTTP status code
- `detail` — human-readable explanation specific to this occurrence
- `instance` — the request URI that triggered the error

Extension: For validation errors (400), a custom `errors` property is added via `ProblemDetail.setProperty()` containing a map of field → message:

```
{
  "type": "about:blank",
  "title": "Validation Failed",
  "status": 400,
  "detail": "Request validation failed",
  "instance": "/api/v1/employees",
  "errors": {
    "email": "must be a valid email address",
    "baseSalary": "must be greater than 0"
  }
}
```

Rationale: RFC 7807 is the industry standard for HTTP error responses. Spring Boot 3+ supports it natively. The `errors` extension is necessary for validation failures — without it, all field errors collapse into a single generic message which is not actionable for the client.

Implementation: A single `@ControllerAdvice` class (`GlobalExceptionHandler`) handles all exceptions and maps them to `ProblemDetail`.

Status code mapping:

Scenario	Exception	Status
Employee not found	`ResourceNotFoundException`	404
Payroll slip not found	`ResourceNotFoundException`	404
DTO field validation failure	`MethodArgumentNotValidException`	400
Duplicate email on create	`DuplicateEmailException`	409
Payroll already exists for period	`PayrollAlreadyExistsException`	409
Payroll already finalized	`PayrollAlreadyFinalizedException`	409
Calculating payroll for INACTIVE employee	`InactiveEmployeeException`	422

TD-002 — Validation split between DTO annotations and service layer

Decision: Validation is applied at two levels depending on whether the rule requires DB context.

Rule:

- **DTO annotation** (`@NotBlank` , `@Email` , `@NotNull` , `@Min` , `@Max` , etc.) — for structural and format rules that are always true regardless of system state. Validated automatically by Spring via `@Valid` on the controller method parameter.
- **Service layer** — for rules that require querying the DB or checking business state.

DTO annotation examples:

- `first_name` , `last_name` — `@NotBlank`
- `email` — `@NotBlank` , `@Email`
- `hire_date` — `@NotNull`
- `department` , `position` — `@NotBlank`
- `base_salary` — `@NotNull` , `@DecimalMin("0.01")`
- `bonus` — `@DecimalMin("0.0")`
- `period_month` — `@Min(1)` , `@Max(12)`
- `period_year` — `@Min(2000)`

Service validation examples:

- Email uniqueness on create → query DB → throw `DuplicateEmailException` → 409
- Employee exists → query DB → throw `ResourceNotFoundException` → 404
- Employee is ACTIVE before payroll calculation → check status field → throw `InactiveEmployeeException` → 422
- Payroll slip already exists for period → query DB → throw `PayrollAlreadyExistsException` → 409
- Payroll slip is DRAFT before finalization → check status field → throw `PayrollAlreadyFinalizedException` → 409

Rationale: DTO annotations catch bad input early, before the service is even called. Service validation handles rules that require state awareness. Mixing the two in the wrong layer leads to either anemic services or bloated DTOs.

TD-003 — Entity ↔ DTO mapping via ModelMapper singleton bean

Decision: ModelMapper is used for entity-to-DTO and DTO-to-entity conversion. A single `ModelMapper` instance is declared as a Spring `@Bean` in a `@Configuration` class and injected wherever mapping is needed.

Rationale: ModelMapper is expensive to instantiate — it scans and caches type mappings at creation time. A singleton bean pays that cost once at startup. Never instantiate with `new ModelMapper()` inside a service or method.

Usage: Services inject `ModelMapper` and call `modelMapper.map(source, TargetClass.class)` . Custom mappings (if needed) are configured once on the bean definition.

TD-004 — Single DTO class per resource (no request/response split)

Decision: One DTO class per resource (e.g., `EmployeeDto` , `PayrollSlipDto`). No separate `EmployeeRequest` / `EmployeeResponse` classes.

Rationale: Keeps the number of classes low for the scope of this exercise. Fields like `id` and `createdAt` that only make sense in responses are simply nullable/ignored on input.

TD-005 — Database: PostgreSQL with Flyway migrations

Decision: PostgreSQL is the database for both development and production. Schema is managed with Flyway — no `ddl-auto=create` or `ddl-auto=update` .

Migration conventions:

- Files in `src/main/resources/db/migration/`
- Named `V{version}__{description}.sql` (e.g., `V1__create_employee_table.sql`)
- Each table gets its own migration file
- Migrations are append-only — never edit an existing migration file

Rationale: Flyway migrations are version-controlled, repeatable, and match how schema changes are managed in production systems. Using `ddl-auto` is acceptable for prototypes but signals lack of production-readiness to evaluators.

TD-006 — Testing: unit tests + integration tests with Testcontainers

Two levels of testing:

Unit tests:

- Target: `PayrollCalculator` and any pure business logic
- Tool: JUnit 5 + Mockito
- No Spring context loaded — plain Java instantiation
- Services are tested with mocked repositories
- Fast, no infrastructure needed

Integration tests:

- Target: full request-to-DB flow per feature
- Tool: `@SpringBootTest` + `MockMvc` + Testcontainers (PostgreSQL container)
- One test class per controller (e.g., `EmployeeControllerIT`, `PayrollControllerIT`)
- Tests make real HTTP calls through MockMvc, hit a real PostgreSQL instance, assert on response body and DB state
- Flyway migrations run automatically on the test container at startup

Rationale: Testcontainers with PostgreSQL catches issues that H2 would miss (type differences, constraint behavior, SQL dialect). `@SpringBootTest` + `MockMvc` tests the full stack in one shot without the fragility of splitting into web/data slices.

TD-007 — Pagination via Spring's Page envelope

Decision: All list endpoints return Spring's native `Page<T>` response envelope. No custom wrapper.

Default parameters: `page=0`, `size=20` if not provided by the caller.

Response shape:

```
{
  "content": [...],
  "pageable": { "pageNumber": 0, "pageSize": 20 },
  "totalElements": 42,
  "totalPages": 3,
  "last": false,
  "first": true
}
```

Rationale: Spring's `Page<T>` is immediately familiar to any Spring developer and requires no additional code. The envelope provides all metadata a client needs to implement pagination UI.

TD-008 — Bulk payroll response shape

Decision: The bulk payroll endpoint returns a summary object with two lists:

```
{
  "created": [
    { "employeeId": 1, "employeeName": "John Doe", "period": "2026-03" },
    { "employeeId": 2, "employeeName": "Jane Smith", "period": "2026-03" }
  ],
  "skipped": [
    { "employeeId": 3, "employeeName": "Bob Martin", "reason": "Payroll already exists for this period" }
  ]
}
```

Rationale: HR needs to know exactly which employees were processed and which were skipped and why. A single count is not enough. The response is a flat summary — it does not embed full payroll slip details (those are retrievable individually via UC-PR-04).

TD-010 — API documentation via Springdoc OpenAPI (Swagger)

Decision: API documentation is auto-generated from code using Springdoc OpenAPI. No manual API contract document maintained separately.

Dependency: `springdoc-openapi-starter-webmvc-ui`

Access: Swagger UI available at `/swagger-ui.html`, OpenAPI JSON at `/v3/api-docs`.

Rationale: Swagger generates documentation that is always in sync with the actual implementation. Manual API docs go stale. Annotations like `@Operation`, `@ApiResponse`, and `@Schema` on controllers and DTOs enrich the generated documentation without duplicating effort.

TD-009 — Audit fields via Spring Data JPA Auditing

Decision: `createdAt` and `updatedAt` on `Employee` are managed automatically via Spring Data JPA Auditing (`@CreatedDate`, `@LastModifiedDate`, `@EnableJpaAuditing`).

Implementation:

- `@EnableJpaAuditing` on the main application class or a `@Configuration` class
- A `@MappedSuperclass` base entity (`BaseEntity`) holds the audit fields and is annotated with `@EntityListeners(AuditingEntityListener.class)`
- `Employee` extends `BaseEntity`
- `PayrollSlip` uses only `calculatedAt` set manually at calculation time — no auditing needed there

Rationale: Eliminates manual `LocalDateTime.now()` calls scattered across service methods. Audit fields are always populated consistently regardless of which service method persists the entity.

API Contract

Base path: `/api/v1`

All error responses follow RFC 7807 (see `06-technical-decisions.md` TD-001). All list endpoints return Spring's `Page<T>` envelope (see TD-007).

Employee

POST /api/v1/employees

Create a new employee.

Request body:

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@company.com",
  "hireDate": "2024-01-15",
  "department": "Engineering",
  "position": "Backend Developer",
  "baseSalary": 5000.00,
  "bonus": 500.00
}
```

Field	Required	Constraint
firstName	yes	not blank
lastName	yes	not blank
email	yes	valid email format, unique
hireDate	yes	not null
department	yes	not blank
position	yes	not blank
baseSalary	yes	> 0
bonus	no	>= 0, defaults to 0

Response: `201 Created`

```
{
  "id": 1,
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@company.com",
  "hireDate": "2024-01-15",
  "department": "Engineering",
  "position": "Backend Developer",
  "baseSalary": 5000.00,
  "bonus": 500.00,
```

```
  "status": "ACTIVE",
  "createdAt": "2026-03-27T10:00:00Z",
  "updatedAt": "2026-03-27T10:00:00Z"
}
```

Error responses:

- `400` — validation failure (invalid field format or missing required field)
- `409` — email already exists

GET /api/v1/employees

List all employees with optional filters. Paginated.

Query parameters:

Parameter	Required	Description
status	no	Filter by `ACTIVE` or `INACTIVE`
department	no	Filter by department (plain string match)
page	no	Page index, default `0`
size	no	Page size, default `20`

Response: `200 OK` — `Page<EmployeeDto>`

GET /api/v1/employees/{id}

Retrieve a single employee by ID.

Response: `200 OK` — same shape as POST response.

Error responses:

- `404` — employee not found

PUT /api/v1/employees/{id}

Full update of an employee's information.

Request body: same fields as POST (all fields required).

Response: `200 OK` — updated employee, same shape as POST response.

Error responses:

- `400` — validation failure
- `404` — employee not found
- `409` — email already taken by another employee

DELETE /api/v1/employees/{id}

Deactivate an employee (soft delete). Sets status to `INACTIVE`.

Response: `204 No Content`

Error responses:

- `404` — employee not found

Payroll

POST /api/v1/employees/{id}/payroll/{year}/{month}

Calculate payroll for a specific employee and period. Creates a `DRAFT` slip.

Path variables:

Variable	Constraint
id	existing employee id
year	>= 2000
month	1–12

Request body: none.

Response: `201 Created`

```
{
  "id": 10,
  "employeeId": 1,
  "periodYear": 2026,
  "periodMonth": 3,
  "status": "DRAFT",
  "calculatedAt": "2026-03-27T10:00:00Z",
  "lineItems": [
    { "type": "EARNING",   "code": "BASE_SALARY",           "label": "Base Salary",
"amount": 5000.00 },
    { "type": "EARNING",   "code": "BONUS",                 "label": "Monthly Bonus",
"amount": 500.00  },
    { "type": "DEDUCTION", "code": "INCOME_TAX",            "label": "Income Tax (20%)",
"amount": 1000.00 },
    { "type": "DEDUCTION", "code": "SOCIAL_CONTRIBUTION",   "label": "Social Contribution (5%)",
"amount": 250.00  }
  ],
  "netSalary": 4250.00
}
```

Error responses:

- `404` — employee not found
- `409` — payroll already exists for this period
- `422` — employee is INACTIVE

POST /api/v1/payroll/bulk

Calculate payroll for all `ACTIVE` employees for a given period. Employees who already have a slip for the period are skipped.

Request body:

```
{
  "year": 2026,
```

```
  "month": 3
}
```

Field	Required	Constraint
year	yes	>= 2000
month	yes	1–12

Response: `200 OK`

```
{
  "created": [
    { "employeeId": 1, "employeeName": "John Doe",    "period": "2026-03" },
    { "employeeId": 2, "employeeName": "Jane Smith",  "period": "2026-03" }
  ],
  "skipped": [
    { "employeeId": 3, "employeeName": "Bob Martin",  "reason": "Payroll already exists for this
period" }
  ]
}
```

Error responses:

- `400` — invalid year or month

POST /api/v1/employees/{id}/payroll/{year}/{month}/finalize

Finalize a `DRAFT` payroll slip. Once finalized, the slip is immutable.

Request body: none.

Response: `200 OK` — finalized slip, same shape as payroll calculation response with `status: "FINALIZED"`.

Error responses:

- `404` — payroll slip not found
- `409` — payroll slip is already finalized

GET /api/v1/employees/{id}/payroll/{year}/{month}

Retrieve a specific payroll slip with full line items breakdown.

Response: `200 OK` — same shape as payroll calculation response.

Error responses:

- `404` — employee not found or no payroll slip for this period

---

GET /api/v1/employees/{id}/payroll

List payroll history for an employee. Paginated, ordered by period descending.

Query parameters:

Parameter	Required	Description
page	no	Page index, default `0`

size	no	Page size, default `20`

Response: `200 OK` — `Page<PayrollSlipDto>` (each slip includes line items and computed net salary).

Error responses:

- `404` — employee not found

Endpoint Summary

Method	Path	Use Case	Status
POST	`/api/v1/employees`	Create employee	201
GET	`/api/v1/employees`	List employees	200
GET	`/api/v1/employees/{id}`	View employee	200
PUT	`/api/v1/employees/{id}`	Update employee	200
DELETE	`/api/v1/employees/{id}`	Deactivate employee	204
POST	`/api/v1/employees/{id}/payroll/{year}/{month}`	Calculate payroll	201
POST	`/api/v1/payroll/bulk`	Bulk calculate payroll	200
POST	`/api/v1/employees/{id}/payroll/{year}/{month}/finalize`	Finalize payroll slip	200
GET	`/api/v1/employees/{id}/payroll/{year}/{month}`	View payroll slip	200
GET	`/api/v1/employees/{id}/payroll`	List payroll history	200

Developer User Stories

Project: Talenteo HR System **Stack:** Spring Boot 3, PostgreSQL, Flyway, ModelMapper, Springdoc OpenAPI, Testcontainers

Each story is atomic and ordered by implementation dependency. The sum of all stories = the complete implementation.

EPIC-1: Project Setup and Infrastructure

> *Goal: A runnable Spring Boot application with database connectivity, Flyway, shared configuration, and audit infrastructure. No feature code yet.*

US-01 — Initialize Spring Boot project with dependencies

As a developer, I want to initialize the Spring Boot project with all required dependencies so the skeleton compiles and runs.

Description: Create a Maven Spring Boot 3 project with the following dependencies:

- `spring-boot-starter-web`
- `spring-boot-starter-data-jpa`
- `spring-boot-starter-validation`
- `postgresql` (runtime)
- `flyway-core`
- `modelmapper`
- `springdoc-openapi-starter-webmvc-ui`
- `lombok`
- `spring-boot-starter-test`
- `testcontainers` (postgresql)

Acceptance Criteria:

- [x] Project compiles with `mvn clean package`
- [x] All dependencies resolve without conflicts
- [x] Main application class exists with `@SpringBootApplication`
- [x] Base package is `com.talenteo.hr`

Technical Notes:

- Spring Boot version: 3.x
- Java version: 17+
- Group: `com.talenteo`, Artifact: `hr`

US-02 — Configure application properties

As a developer, I want to configure `application.yml` with datasource, JPA, Flyway, and Springdoc settings so the application connects to PostgreSQL and serves the Swagger UI.

Description: Create `src/main/resources/application.yml` with:

- PostgreSQL datasource (url, username, password)
- `spring.jpa.hibernate.ddl-auto=validate` (Flyway manages schema)
- `spring.flyway.enabled=true`
- `springdoc.swagger-ui.path=/swagger-ui.html`
- `springdoc.api-docs.path=/v3/api-docs`

Create `src/test/resources/application-test.yml` (Testcontainers overrides datasource at runtime — minimal config needed).

Acceptance Criteria:

- ☑ Application starts without errors against a running PostgreSQL instance
- ☑ `ddl-auto=validate` is set (Flyway owns the schema, not Hibernate)
- ☑ Swagger UI accessible at `/swagger-ui.html` when app is running

Technical Notes:

- TD-005: Flyway manages schema, never `ddl-auto=create` or `update`
- TD-010: Springdoc OpenAPI

US-03 — Flyway migration V1: create employees table

As a developer, I want to write the first Flyway migration that creates the `employees` table so the schema is version-controlled from day one.

Description: Create `src/main/resources/db/migration/V1__create_employee_table.sql`:

```sql
CREATE TABLE employees (
    id BIGSERIAL PRIMARY KEY,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL,
    hire_date DATE NOT NULL,
    department VARCHAR(100) NOT NULL,
    position VARCHAR(100) NOT NULL,
    base_salary DECIMAL(12,2) NOT NULL,
    bonus DECIMAL(12,2) NOT NULL DEFAULT 0,
    status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    CONSTRAINT uk_employees_email UNIQUE (email),
    CONSTRAINT chk_base_salary CHECK (base_salary > 0),
    CONSTRAINT chk_bonus CHECK (bonus >= 0)
);
```

Acceptance Criteria:

- ☑ Migration runs successfully on a clean PostgreSQL instance
- ☑ Unique constraint on `email` exists
- ☑ Check constraints on `base_salary` and `bonus` exist
- ☑ Migration is named exactly `V1__create_employee_table.sql`

Technical Notes:

- TD-005: append-only migrations, never edit after applying
- DM-001: department and position are plain VARCHAR, no FK
- DM-002: status is VARCHAR storing enum values

US-04 — Flyway migration V2: create payroll_slip table

As a developer, I want to write the second Flyway migration that creates the `payroll_slip` table with its unique constraint so duplicate payrolls are prevented at the DB level.

Description: Create `src/main/resources/db/migration/V2__create_payroll_slip_table.sql`:

```sql
CREATE TABLE payroll_slip (
    id BIGSERIAL PRIMARY KEY,
    employee_id BIGINT NOT NULL,
    period_year SMALLINT NOT NULL,
    period_month SMALLINT NOT NULL,
    net_salary DECIMAL(12,2) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'DRAFT',
    calculated_at TIMESTAMP NOT NULL,
    CONSTRAINT fk_payroll_slip_employee FOREIGN KEY (employee_id) REFERENCES employees(id),
    CONSTRAINT uk_payroll_slip_period UNIQUE (employee_id, period_year, period_month),
    CONSTRAINT chk_period_month CHECK (period_month BETWEEN 1 AND 12),
    CONSTRAINT chk_period_year CHECK (period_year >= 2000)
);
```

Acceptance Criteria:

- ☑ Migration runs successfully after V1
- ☑ Unique constraint on `(employee_id, period_year, period_month)` exists
- ☑ FK to `employees.id` exists
- ☑ Check constraints on year and month exist

Technical Notes:

- DM-004: `net_salary` stored (denormalized), computed at calculation time
- DM-005: immutability enforced at DB level via unique constraint
- UC-003: duplicate returns 409

US-05 — Flyway migration V3: create payroll_line_item table

As a developer, I want to write the third Flyway migration that creates the `payroll_line_item` table so each payroll breakdown can be stored as immutable rows.

Description: Create `src/main/resources/db/migration/V3__create_payroll_line_item_table.sql`:

```sql
CREATE TABLE payroll_line_item (
    id BIGSERIAL PRIMARY KEY,
    payroll_slip_id BIGINT NOT NULL,
    type VARCHAR(20) NOT NULL,
    code VARCHAR(50) NOT NULL,
    label VARCHAR(150) NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    CONSTRAINT fk_line_item_slip FOREIGN KEY (payroll_slip_id) REFERENCES payroll_slip(id)
);
```

Acceptance Criteria:

- ☑ Migration runs successfully after V2
- ☑ FK to `payroll_slip.id` exists
- ☑ `amount`, `label`, `type`, `code` are all NOT NULL

Technical Notes:

- DM-003: line items are the backbone of the header + items model
- DM-006: label is a snapshot (e.g., "Income Tax (20%)") — not derived at query time

---

US-06 — BaseEntity with JPA Auditing

As a developer, I want to implement a `BaseEntity` superclass with automatic audit fields so `createdAt` and `updatedAt` are populated consistently without manual code.

Description: Create `com.talenteo.hr.common.config.BaseEntity`:

```
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public abstract class BaseEntity {
    @CreatedDate
    @Column(nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    @Column(nullable = false)
    private LocalDateTime updatedAt;
}
```

Enable auditing on the main application class or a `@Configuration` class with `@EnableJpaAuditing`.

Acceptance Criteria:

- ☑ `BaseEntity` is a `@MappedSuperclass` with `@EntityListeners(AuditingEntityListener.class)`
- ☑ `createdAt` is non-updatable
- ☑ `@EnableJpaAuditing` is present
- ☑ `Employee` will extend `BaseEntity` (verified in US-11)

Technical Notes:

- TD-009: JPA Auditing eliminates scattered `LocalDateTime.now()` calls
- Package: `com.talenteo.hr.common.config`

US-07 — ModelMapper singleton bean

As a developer, I want to declare a singleton `ModelMapper` bean so all entity-DTO conversions share one pre-initialized instance.

Description: Create `com.talenteo.hr.common.config.AppConfig`:

```
@Configuration
public class AppConfig {
    @Bean
    public ModelMapper modelMapper() {
        return new ModelMapper();
    }
}
```

Acceptance Criteria:

- ☑ `ModelMapper` is declared as a `@Bean` in a `@Configuration` class
- ☑ Only one `ModelMapper` bean exists in the application context
- ☑ No `new ModelMapper()` calls appear outside this class

Technical Notes:

- TD-003: ModelMapper is expensive to instantiate — singleton only
- Package: `com.talenteo.hr.common.config`

US-08 — Springdoc OpenAPI configuration

As a developer, I want to configure Springdoc OpenAPI with project metadata so the Swagger UI displays meaningful information about the API.

Description: Create `com.talenteo.hr.common.config.OpenApiConfig`:

```
@Configuration
public class OpenApiConfig {
    @Bean
    public OpenAPI hrApiOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("Talenteo HR API")
                .description("Employee Management and Payroll Processing")
                .version("1.0.0"));
    }
}
```

Acceptance Criteria:

- [x] Swagger UI accessible at `/swagger-ui.html`
- [x] OpenAPI JSON accessible at `/v3/api-docs`
- [x] API title, description, and version are displayed in the UI

Technical Notes:

- TD-010: Springdoc generates docs from code — no manual API doc maintenance
- Package: `com.talenteo.hr.common.config`

EPIC-2: Common / Cross-Cutting Infrastructure

> *Goal: Exception classes and global error handler in place before any feature code. Every service story depends on these.*

US-09 — Custom exception hierarchy

As a developer, I want to define all custom exceptions so services can throw meaningful, typed errors that the global handler maps to correct HTTP responses.

Description: Create the following in `com.talenteo.hr.common.exception`:

```
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) { super(message); }
}

public class DuplicateEmailException extends RuntimeException {
    public DuplicateEmailException(String message) { super(message); }
}

public class PayrollAlreadyExistsException extends RuntimeException {
    public PayrollAlreadyExistsException(String message) { super(message); }
}
```

```java
public class PayrollAlreadyFinalizedException extends RuntimeException {
    public PayrollAlreadyFinalizedException(String message) { super(message); }
}

public class InactiveEmployeeException extends RuntimeException {
    public InactiveEmployeeException(String message) { super(message); }
}
```

Acceptance Criteria:

- ☑ All 5 exception classes exist in `com.talenteo.hr.common.exception`
- ☑ All extend `RuntimeException`
- ☑ All accept a `String message` constructor parameter

Technical Notes:

- TD-001: status code mapping — ResourceNotFoundException → 404, DuplicateEmail/PayrollAlreadyExists/PayrollAlreadyFinalized → 409, InactiveEmployee → 422

US-10 — GlobalExceptionHandler

As a developer, I want to implement a `@ControllerAdvice` that maps all exceptions to RFC 7807 `ProblemDetail` responses so every error follows a consistent structure.

Description: Create `com.talenteo.hr.common.exception.GlobalExceptionHandler` annotated with `@ControllerAdvice`.

Handle:

- `ResourceNotFoundException` → `ProblemDetail` with status 404
- `DuplicateEmailException` → 409
- `PayrollAlreadyExistsException` → 409
- `PayrollAlreadyFinalizedException` → 409
- `InactiveEmployeeException` → 422
- `MethodArgumentNotValidException` → 400 with `errors` property (map of field → message) added via `problemDetail.setProperty("errors", fieldErrors)`

Acceptance Criteria:

- ☑ All 5 custom exceptions are handled and return correct status codes
- ☑ `MethodArgumentNotValidException` returns 400 with `errors` map in the response body
- ☑ All responses use Spring's `ProblemDetail` (not a custom class)
- ☑ No exception leaks as a 500 for the mapped scenarios

Technical Notes:

- TD-001: RFC 7807, `ProblemDetail.forStatusAndDetail(HttpStatus, String)`
- Package: `com.talenteo.hr.common.exception`

EPIC-3: Employee Management

> *Goal: Full employee feature end-to-end covering all 5 use cases (UC-EM-01 to UC-EM-05) and their error paths.*

US-11 — Employee JPA entity

As a developer, I want to implement the `Employee` JPA entity extending `BaseEntity` so it maps correctly to the `employees` table.

Description: Create `com.talenteo.hr.employee.domain.Employee` extending `BaseEntity`:

- `@Entity`, `@Table(name = "employees")`
- Fields: `id` (BIGSERIAL), `firstName`, `lastName`, `email`, `hireDate`, `department`, `position`, `baseSalary`, `bonus`, `status`
- `EmployeeStatus` enum: `ACTIVE`, `INACTIVE` stored as `@Enumerated(EnumType.STRING)`
- `baseSalary` and `bonus` as `BigDecimal`

Acceptance Criteria:

- ☑ Entity maps to `employees` table without Hibernate errors
- ☑ `status` is stored as string (`ACTIVE` / `INACTIVE`)
- ☑ `baseSalary` and `bonus` are `BigDecimal` (never `double` or `float`)
- ☑ Extends `BaseEntity` (inherits `createdAt`, `updatedAt`)

Technical Notes:

- Package: `com.talenteo.hr.employee.domain`
- DM-002: soft delete via status field
- Always use `BigDecimal` for money — never `double`

US-12 — EmployeeRepository

As a developer, I want to implement `EmployeeRepository` with query methods for filtered, paginated listing so UC-EM-03 can filter by status and department.

Description: Create `com.talenteo.hr.employee.repository.EmployeeRepository` extending `JpaRepository<Employee, Long>`.

Add a query method supporting optional filters:

```
Page<Employee> findByStatusAndDepartmentContainingIgnoreCase(
    EmployeeStatus status, String department, Pageable pageable);
```

Or use a `@Query` with `JPQL` for more flexible optional filtering:

```
@Query("SELECT e FROM Employee e WHERE (:status IS NULL OR e.status = :status) AND (:department
IS NULL OR LOWER(e.department) LIKE LOWER(CONCAT('%', :department, '%')))")
Page<Employee> findWithFilters(@Param("status") EmployeeStatus status, @Param("department")
String department, Pageable pageable);
```

Acceptance Criteria:

- ☑ Repository extends `JpaRepository<Employee, Long>`
- ☑ Supports paginated listing with optional status filter
- ☑ Supports optional department filter (case-insensitive partial match)
- ☑ No business logic in the repository

Technical Notes:

- Package: `com.talenteo.hr.employee.repository`
- UC-EM-03: filter by status and department, paginated

US-13 — EmployeeDto

As a developer, I want to define `EmployeeDto` with Bean Validation annotations so input is validated automatically before reaching the service.

Description: Create `com.talenteo.hr.employee.dto.EmployeeDto` with all fields and annotations:

```
public class EmployeeDto {
    private Long id;                        // response only, null on input

    @NotBlank
    private String firstName;

    @NotBlank
    private String lastName;

    @NotBlank @Email
    private String email;

    @NotNull
    private LocalDate hireDate;

    @NotBlank
    private String department;

    @NotBlank
    private String position;

    @NotNull @DecimalMin("0.01")
    private BigDecimal baseSalary;

    @DecimalMin("0.0")
    private BigDecimal bonus;

    private EmployeeStatus status;          // response only, ignored on input
    private LocalDateTime createdAt;        // response only
    private LocalDateTime updatedAt;        // response only
}
```

Acceptance Criteria:

- ☑ All input fields have appropriate validation annotations
- ☑ `id`, `status`, `createdAt`, `updatedAt` are present but not validated (response fields)
- ☑ `baseSalary` minimum is `0.01` (strictly positive)
- ☑ `bonus` minimum is `0.0` (zero allowed)

Technical Notes:

- TD-002: DTO annotations for structural/format rules
- TD-004: single DTO class — no request/response split

US-14 — EmployeeService: create

As a developer, I want to implement the `createEmployee` method in `EmployeeService` so UC-EM-01 is fulfilled with email uniqueness enforcement.

Description: Implement in `com.talenteo.hr.employee.service.EmployeeService`:

```
createEmployee(EmployeeDto dto) → EmployeeDto
```

Steps:

1. Check email uniqueness via repository — throw `DuplicateEmailException` if taken
2. Map DTO to entity via `ModelMapper`
3. Set `status = ACTIVE`
4. Save and return mapped response DTO

Acceptance Criteria:

- [x] Returns created employee DTO with `id`, `status=ACTIVE`, `createdAt`, `updatedAt` populated
- [x] Throws `DuplicateEmailException` if email already exists
- [x] Does not set `id` manually (auto-generated)
- [x] `bonus` defaults to `BigDecimal.ZERO` if null

Technical Notes:

- Package: `com.talenteo.hr.employee.service`
- UC-EM-01 precondition: email must be unique

US-15 — EmployeeService: read and list

As a developer, I want to implement `getEmployeeById` and `listEmployees` in `EmployeeService` so UC-EM-02 and UC-EM-03 are fulfilled.

Description: Implement:

```
getEmployeeById(Long id) → EmployeeDto
listEmployees(EmployeeStatus status, String department, Pageable pageable) → Page<EmployeeDto>
```

`getEmployeeById`: fetch by id, throw `ResourceNotFoundException` if absent, return mapped DTO.

`listEmployees`: delegate to repository with optional filters, map each entity to DTO.

Acceptance Criteria:

- [x] `getEmployeeById` throws `ResourceNotFoundException` for unknown id
- [x] `listEmployees` returns paginated results
- [x] Both `status` and `department` filters are optional (null = no filter)
- [x] `department` filter is case-insensitive

Technical Notes:

- UC-EM-02, UC-EM-03
- TD-007: `Page<T>` returned directly

US-16 — EmployeeService: update

As a developer, I want to implement `updateEmployee` in `EmployeeService` so UC-EM-04 is fulfilled with correct email uniqueness validation.

Description: Implement:

```
updateEmployee(Long id, EmployeeDto dto) → EmployeeDto
```

Steps:

1. Fetch existing employee — throw `ResourceNotFoundException` if absent

2. If email changed: check new email is not taken by *another* employee — throw `DuplicateEmailException` if so
3. Map all fields from DTO onto the existing entity
4. Save and return updated DTO

Acceptance Criteria:

- ☑ Throws `ResourceNotFoundException` if employee not found
- ☑ Throws `DuplicateEmailException` if email is taken by a *different* employee
- ☑ Does NOT throw on email uniqueness if the email belongs to the same employee (unchanged email)
- ☑ Updating salary does not affect past payroll slips (they are already snapshotted)

Technical Notes:

- UC-EM-04
- Key edge case: email uniqueness check must exclude the current employee's own id

---

US-17 — EmployeeService: deactivate

As a developer, I want to implement `deactivateEmployee` in `EmployeeService` so UC-EM-05 is fulfilled with soft delete.

Description: Implement:

```
deactivateEmployee(Long id) → void
```

Steps:

1. Fetch employee — throw `ResourceNotFoundException` if absent
2. Set `status = INACTIVE`
3. Save

Acceptance Criteria:

- ☑ Throws `ResourceNotFoundException` if employee not found
- ☑ Sets `status = INACTIVE` — does not delete the record
- ☑ Payroll history is unaffected
- ☑ `updatedAt` is updated (handled by JPA Auditing)

Technical Notes:

- DM-002: soft delete
- UC-EM-05

US-18 — EmployeeController

As a developer, I want to implement `EmployeeController` with all 5 endpoints wired to `EmployeeService` so the employee API is complete and documented via Swagger.

Description: Create `com.talenteo.hr.employee.controller.EmployeeController` with `@RestController`, `@RequestMapping("/api/v1/employees")`:

Method	Path	Service call	Status
POST	`/`	`createEmployee`	201
GET	`/`	`listEmployees`	200
GET	`/{id}`	`getEmployeeById`	200

PUT	`/{id}`	`updateEmployee`	200
DELETE	`/{id}`	`deactivateEmployee`	204

- Use `@Valid` on request body parameters
- Use `ResponseEntity` to set correct status codes
- Annotate with `@Operation` and `@ApiResponse` for Swagger

Acceptance Criteria:

- [x] All 5 endpoints are present and return correct HTTP status codes
- [x] `@Valid` is applied on POST and PUT request bodies
- [x] List endpoint accepts `status`, `department`, `page`, `size` query params
- [x] DELETE returns `204 No Content` (no body)
- [x] Swagger UI displays all 5 endpoints with descriptions

Technical Notes:

- Package: `com.talenteo.hr.employee.controller`
- TD-010: `@Operation`, `@ApiResponse` Swagger annotations

EPIC-4: Payroll Feature

> *Goal: Core payroll feature covering single-employee use cases (UC-PR-01, UC-PR-03 to UC-PR-05). Bulk processing is in EPIC-4.5.*

US-19 — PayrollSlip and PayrollLineItem JPA entities

As a developer, I want to implement the `PayrollSlip` and `PayrollLineItem` JPA entities with their enums and relationship so the payroll data model is correctly mapped.

Description: Create `com.talenteo.hr.payroll.domain.PayrollSlip`:

- `@Entity`, `@Table(name = "payroll_slip")`
- Fields: `id`, `employee` (ManyToOne), `periodYear`, `periodMonth`, `netSalary` (BigDecimal), `status` (PayrollStatus enum), `calculatedAt`
- `PayrollStatus` enum: `DRAFT`, `FINALIZED`
- `@OneToMany(mappedBy = "payrollSlip", cascade = CascadeType.ALL, fetch = FetchType.EAGER)` for `lineItems`

Create `com.talenteo.hr.payroll.domain.PayrollLineItem`:

- `@Entity`, `@Table(name = "payroll_line_item")`
- Fields: `id`, `payrollSlip` (ManyToOne), `type` (LineItemType enum), `code` (LineItemCode enum), `label`, `amount` (BigDecimal)
- `LineItemType` enum: `EARNING`, `DEDUCTION`
- `LineItemCode` enum: `BASE_SALARY`, `BONUS`, `INCOME_TAX`, `SOCIAL_CONTRIBUTION`, `HIGH_SALARY_SURCHARGE`

Acceptance Criteria:

- [x] Both entities map to their tables without Hibernate errors
- [x] `@OneToMany` on `PayrollSlip.lineItems` uses `CascadeType.ALL`
- [x] All monetary fields are `BigDecimal`
- [x] Enums are stored as `STRING`

Technical Notes:

- Package: `com.talenteo.hr.payroll.domain`
- DM-003: header + line items model
- `PayrollSlip` does NOT extend `BaseEntity` — uses `calculatedAt` set manually

US-20 — PayrollSlipRepository

As a developer, I want to implement `PayrollSlipRepository` with query methods for duplicate detection and paginated history so all payroll read/write use cases are supported.

Description: Create `com.talenteo.hr.payroll.repository.PayrollSlipRepository` extending `JpaRepository<PayrollSlip, Long>`:

```
Optional<PayrollSlip> findByEmployeeIdAndPeriodYearAndPeriodMonth(
    Long employeeId, int year, int month);

Page<PayrollSlip> findByEmployeeIdOrderByPeriodYearDescPeriodMonthDesc(
    Long employeeId, Pageable pageable);
```

Acceptance Criteria:

- ☑ `findByEmployeeIdAndPeriodYearAndPeriodMonth` returns `Optional` for duplicate detection
- ☑ `findByEmployeeId...` supports paginated history ordered by period descending
- ☑ No business logic in the repository

Technical Notes:

- Package: `com.talenteo.hr.payroll.repository`
- UC-PR-01: duplicate guard, UC-PR-04: view slip, UC-PR-05: history

---

US-21 — PayrollLineItemRepository

As a developer, I want to implement `PayrollLineItemRepository` so line items can be persisted independently if needed.

Description: Create `com.talenteo.hr.payroll.repository.PayrollLineItemRepository` extending `JpaRepository<PayrollLineItem, Long>`. No custom query methods needed — line items are read through the `PayrollSlip.lineItems` collection.

Acceptance Criteria:

- ☑ Repository extends `JpaRepository<PayrollLineItem, Long>`
- ☑ No custom methods needed at this stage

Technical Notes:

- Package: `com.talenteo.hr.payroll.repository`

US-22 — Payroll DTOs

As a developer, I want to define all payroll DTOs so requests and responses are cleanly typed and validated.

Description: Create in `com.talenteo.hr.payroll.dto`:

`BulkPayrollRequest`:

- `@Min(2000) int year`
- `@Min(1) @Max(12) int month`

`LineItemDto`:

- `LineItemType type`, `LineItemCode code`, `String label`, `BigDecimal amount`

`PayrollSlipDto`:

- `Long id`, `Long employeeId`, `int periodYear`, `int periodMonth`
- `BigDecimal netSalary`, `PayrollStatus status`, `LocalDateTime calculatedAt`
- `List<LineItemDto> lineItems`

`BulkPayrollResult`:

- `List<BulkCreatedEntry> created` — each: `employeeId`, `employeeName`, `String period` (e.g., "2026-03")
- `List<BulkSkippedEntry> skipped` — each: `employeeId`, `employeeName`, `String reason`

Acceptance Criteria:

- ☑ `BulkPayrollRequest` has validation annotations on `year` and `month`
- ☑ `PayrollSlipDto` includes `lineItems` list
- ☑ `BulkPayrollResult` has both `created` and `skipped` lists

Technical Notes:

- TD-004: single DTO class per resource
- TD-008: bulk response shape

US-23 — PayrollCalculator

As a developer, I want to implement `PayrollCalculator` as a pure calculation component that reads rates from config so business rules are isolated, testable, and changeable without code modifications.

Description: Create `com.talenteo.hr.payroll.config.PayrollProperties` as a `@ConfigurationProperties(prefix = "payroll.rules")` bean:

- `incomeTaxRate` (BigDecimal)
- `socialContributionRate` (BigDecimal)
- `highSalaryThreshold` (BigDecimal)
- `highSalaryRatesSurchargeRate` (BigDecimal)

Add to `application.yml`:

```yaml
payroll:
  rules:
    income-tax-rate: 0.20
    social-contribution-rate: 0.05
    high-salary-threshold: 5000.00
    high-salary-surcharge-rate: 0.05
```

Create `com.talenteo.hr.payroll.service.PayrollCalculator` as a `@Component` injecting `PayrollProperties`.

Method signature:

```java
public List<PayrollLineItem> calculate(BigDecimal baseSalary, BigDecimal bonus)
```

Applies 5 rules using rates from `PayrollProperties`:

1. `BASE_SALARY` (EARNING) = `baseSalary`
2. `BONUS` (EARNING) = `bonus` — only if `bonus > 0`
3. `INCOME_TAX` (DEDUCTION) = `baseSalary × incomeTaxRate`
4. `SOCIAL_CONTRIBUTION` (DEDUCTION) = `baseSalary × socialContributionRate`

5. `HIGH_SALARY_SURCHARGE` (DEDUCTION) = `baseSalary × highSalaryRatesSurchargeRate` — only if `baseSalary > highSalaryThreshold`

Returns detached `PayrollLineItem` objects — no `id` or `payrollSlip` set. `PayrollService` sets the slip reference before persisting.

Acceptance Criteria:

- [x] No repository, no DB access in this class
- [x] Rates injected via `PayrollProperties` — no magic numbers in the calculator
- [x] `BONUS` line item excluded when `bonus` is zero or null
- [x] `HIGH_SALARY_SURCHARGE` only included when `baseSalary > highSalaryThreshold`
- [x] All `BigDecimal` calculations use `setScale(2, RoundingMode.HALF_UP)`
- [x] Label built dynamically from config value (e.g., `"Income Tax (20%)"`)

Technical Notes:

- Package: `com.talenteo.hr.payroll.service` (calculator), `com.talenteo.hr.payroll.config` (properties)
- DM-007 updated: rates from `application.yml` via `@ConfigurationProperties`, not hardcoded
- This class gets its own dedicated unit test in US-29

US-24 — PayrollService: calculate single

As a developer, I want to implement `calculatePayroll` in `PayrollService` so UC-PR-01 is fulfilled with all guards and the snapshot pattern.

Description: Implement in `com.talenteo.hr.payroll.service.PayrollService`:

```
calculatePayroll(Long employeeId, int year, int month) → PayrollSlipDto
```

Steps:

1. Fetch employee — throw `ResourceNotFoundException` if absent
2. Guard: employee must be `ACTIVE` — throw `InactiveEmployeeException` if not
3. Guard: no existing slip for period — throw `PayrollAlreadyExistsException` if exists
4. Call `PayrollCalculator.calculate(employee.baseSalary, employee.bonus)` — snapshots values
5. Compute `netSalary = SUM(EARNINGS) - SUM(DEDUCTIONS)`
6. Build and persist `PayrollSlip` with status `DRAFT`, `calculatedAt = now()`
7. Associate and persist line items
8. Return mapped `PayrollSlipDto`

Acceptance Criteria:

- [x] Throws `ResourceNotFoundException` for unknown employee
- [x] Throws `InactiveEmployeeException` for `INACTIVE` employee (422)
- [x] Throws `PayrollAlreadyExistsException` for duplicate period (409)
- [x] `netSalary` is correctly computed and stored on the slip
- [x] Line items are persisted with correct type, code, label, and amount
- [x] Past and future periods are accepted (no date restriction)

Technical Notes:

- UC-PR-01, DM-004, DM-006
- Package: `com.talenteo.hr.payroll.service`

US-25.5 — PayrollService: recalculate

As a developer, I want to implement `recalculatePayroll` in `PayrollService` so HR can correct a DRAFT slip before finalizing.

Description: Implement:

```
recalculatePayroll(Long employeeId, int year, int month) → PayrollSlipDto
```

Steps:

1. Fetch slip by `(employeeId, year, month)` — throw `ResourceNotFoundException` if absent
2. Guard: status must be `DRAFT` — throw `PayrollAlreadyFinalizedException` if `FINALIZED`
3. Delete existing slip (cascades to line items)
4. Call `calculatePayroll(employeeId, year, month)` — fresh calculation from current salary/bonus and current config rules
5. Return new `PayrollSlipDto`

Acceptance Criteria:

- [x] Throws `ResourceNotFoundException` if slip not found (404)
- [x] Throws `PayrollAlreadyFinalizedException` if slip is FINALIZED (409)
- [x] Old line items are deleted and replaced with fresh ones
- [x] New slip reflects current employee salary/bonus at recalculation time
- [x] Returns new slip with status `DRAFT`

Technical Notes:

- UC-005: recalculation only allowed on DRAFT slips
- Reuses `calculatePayroll()` internally — no duplicated logic
- DB unique constraint requires deletion before re-insert

US-26 — PayrollService: finalize

As a developer, I want to implement `finalizePayroll` in `PayrollService` so UC-PR-03 is fulfilled with immutability enforcement.

Description: Implement:

```
finalizePayroll(Long employeeId, int year, int month) → PayrollSlipDto
```

Steps:

1. Fetch slip by `(employeeId, year, month)` — throw `ResourceNotFoundException` if absent
2. Guard: status must be `DRAFT` — throw `PayrollAlreadyFinalizedException` if already `FINALIZED`
3. Set `status = FINALIZED`
4. Save and return updated `PayrollSlipDto`

Acceptance Criteria:

- [x] Throws `ResourceNotFoundException` if slip not found (404)
- [x] Throws `PayrollAlreadyFinalizedException` if already finalized (409)
- [x] Returns slip with `status = FINALIZED`
- [x] Line items are unchanged after finalization

Technical Notes:

- UC-PR-03, DM-005, UC-001

US-27 — PayrollService: read and list history

As a developer, I want to implement `getPayrollSlip` and `listPayrollHistory` in `PayrollService` so UC-PR-04 and UC-PR-05 are fulfilled.

Description: Implement:

```
getPayrollSlip(Long employeeId, int year, int month) → PayrollSlipDto
listPayrollHistory(Long employeeId, Pageable pageable) → Page<PayrollSlipDto>
```

`getPayrollSlip` : fetch by `(employeeId, year, month)` — throw `ResourceNotFoundException` if absent. Return full DTO with line items.

`listPayrollHistory` : verify employee exists (throw `ResourceNotFoundException` if not), fetch paginated slips ordered by period descending.

Acceptance Criteria:

- ☑ `getPayrollSlip` throws `ResourceNotFoundException` if slip not found
- ☑ `listPayrollHistory` throws `ResourceNotFoundException` if employee not found
- ☑ History is ordered by `periodYear DESC` , `periodMonth DESC`
- ☑ Both return `lineItems` in the DTO

Technical Notes:

- UC-PR-04, UC-PR-05

US-28 — PayrollController

As a developer, I want to implement `PayrollController` with all 5 payroll endpoints wired to `PayrollService` so the payroll API is complete and documented via Swagger.

Description: Create `com.talenteo.hr.payroll.controller.PayrollController` with `@RestController` :

Method	Path	Service call	Status
POST	`/api/v1/employees/{id}/payroll/{year}/{month}`	`calculatePayroll`	201
PUT	`/api/v1/employees/{id}/payroll/{year}/{month}/recalculate`	`recalculatePayroll`	200
POST	`/api/v1/employees/{id}/payroll/{year}/{month}/finalize`	`finalizePayroll`	200
GET	`/api/v1/employees/{id}/payroll/{year}/{month}`	`getPayrollSlip`	200
GET	`/api/v1/employees/{id}/payroll`	`listPayrollHistory`	200

- `@Min(2000)` on `{year}` path variable, `@Min(1) @Max(12)` on `{month}`
- `@Operation` and `@ApiResponse` Swagger annotations on each endpoint

Acceptance Criteria:

- ☑ All 5 endpoints present with correct HTTP status codes
- ☑ Path variable constraints are validated (`@Validated` on controller class)
- ☑ Swagger UI displays all 5 payroll endpoints

Technical Notes:

- Package: `com.talenteo.hr.payroll.controller`
- TD-010: Swagger annotations

EPIC-4.5: Bulk Payroll Processing

> *Goal: Bulk payroll calculation for all active employees in a given period (UC-PR-02). Standalone epic because it crosses the single-employee boundary and has its own endpoint.*

US-25 — PayrollService: bulk calculate

As a developer, I want to implement `bulkCalculatePayroll` in `PayrollService` so UC-PR-02 is fulfilled with correct skip behavior.

Description: Implement:

```
bulkCalculatePayroll(int year, int month) → BulkPayrollResult
```

Steps:

1. Fetch all `ACTIVE` employees
2. For each employee:
   - If slip already exists for period → add to `skipped` list with reason
   - Otherwise → calculate payroll (reuse core logic from US-24), add to `created` list
3. Return `BulkPayrollResult`

Acceptance Criteria:

- ☑ Only processes `ACTIVE` employees
- ☑ Employees with existing slips are added to `skipped` — processing continues for the rest
- ☑ `created` list contains `employeeId`, `employeeName`, `period` (formatted as "YYYY-MM")
- ☑ `skipped` list contains `employeeId`, `employeeName`, `reason`
- ☑ Returns 200 even if all employees were skipped

Technical Notes:

- UC-PR-02, UC-004 conflict behavior

US-28.5 — PayrollController: bulk endpoint

As a developer, I want to add the bulk payroll endpoint to `PayrollController` so UC-PR-02 is reachable via the API.

Description: Add to the existing `PayrollController`:

Method	Path	Service call	Status
POST	`/api/v1/payroll/bulk`	`bulkCalculatePayroll`	200

- `@Valid` on `BulkPayrollRequest` body
- `@Operation` and `@ApiResponse` Swagger annotations

Acceptance Criteria:

- ☑ Endpoint returns 200 with `BulkPayrollResult`
- ☑ `@Valid` enforces year/month constraints
- ☑ Swagger UI displays the bulk endpoint

Technical Notes:

- `BulkPayrollRequest` already defined in US-22

EPIC-5: Testing

> *Goal: Full test coverage across all layers. Unit tests first, then integration tests.*

US-29 — Unit test: PayrollCalculator

As a developer, I want to unit test `PayrollCalculator` in isolation so every calculation rule and edge case is verified without infrastructure.

Description: Create `com.talenteo.hr.payroll.service.PayrollCalculatorTest` using JUnit 5. No Spring context.

Test cases:

1. Base salary only (no bonus) below 5000 → BASE_SALARY + INCOME_TAX + SOCIAL_CONTRIBUTION (3 items)
2. Base salary + bonus below 5000 → BASE_SALARY + BONUS + INCOME_TAX + SOCIAL_CONTRIBUTION (4 items)
3. Base salary above 5000, no bonus → includes HIGH_SALARY_SURCHARGE (4 items)
4. Base salary above 5000 + bonus → all 5 line items
5. Bonus = 0 → BONUS line item excluded
6. Exact amounts: salary=6000 → INCOME_TAX=1200, SOCIAL_CONTRIBUTION=300, SURCHARGE=300, net=(6000-1800)=4200
7. Salary exactly 5000 → no surcharge (boundary: > 5000, not >=)

Acceptance Criteria:

- [ ] All 7 test cases pass
- [ ] `PayrollProperties` instantiated directly with test values — no Spring context needed
- [ ] Amounts verified with exact `BigDecimal` equality
- [ ] Boundary condition at salary=5000 is explicitly tested

Technical Notes:

- DM-007: rates from `PayrollProperties` (config), not magic numbers
- Instantiate `PayrollCalculator` with a manually built `PayrollProperties` object in tests
- Use `assertThat(result).hasSize(n)` and verify each line item

US-30 — Unit test: EmployeeService

As a developer, I want to unit test `EmployeeService` with mocked dependencies so all service-layer business rules are verified in isolation.

Description: Create `com.talenteo.hr.employee.service.EmployeeServiceTest` using JUnit 5 + Mockito.

Test cases:

1. `createEmployee` — happy path: repository saves and returns mapped DTO with status ACTIVE
2. `createEmployee` — duplicate email: `findByEmail` returns existing → throws `DuplicateEmailException`
3. `getEmployeeById` — not found: repository returns empty → throws `ResourceNotFoundException`
4. `updateEmployee` — email taken by another employee → throws `DuplicateEmailException`
5. `updateEmployee` — same email as own employee → succeeds (no false positive)
6. `deactivateEmployee` — sets status to INACTIVE
7. `deactivateEmployee` — not found → throws `ResourceNotFoundException`

Acceptance Criteria:

- [ ] All 7 test cases pass
- [ ] `EmployeeRepository` and `ModelMapper` are mocked
- [ ] No Spring context loaded

- [ ] Exception types are exactly the expected custom exceptions

---

US-31 — Unit test: PayrollService

As a developer, I want to unit test `PayrollService` with mocked dependencies so all orchestration logic and guards are verified in isolation.

Description: Create `com.talenteo.hr.payroll.service.PayrollServiceTest` using JUnit 5 + Mockito.

Test cases:

1. `calculatePayroll` — employee not found → throws `ResourceNotFoundException`
2. `calculatePayroll` — employee is INACTIVE → throws `InactiveEmployeeException`
3. `calculatePayroll` — slip already exists → throws `PayrollAlreadyExistsException`
4. `calculatePayroll` — happy path: slip created with correct netSalary and line items
5. `finalizePayroll` — slip not found → throws `ResourceNotFoundException`
6. `finalizePayroll` — already finalized → throws `PayrollAlreadyFinalizedException`
7. `bulkCalculatePayroll` — mixed: 2 created, 1 skipped → result lists correctly populated

Acceptance Criteria:

- [ ] All 7 test cases pass
- [ ] All dependencies mocked (EmployeeRepository, PayrollSlipRepository, PayrollCalculator, ModelMapper)
- [ ] No Spring context loaded

US-32 — Testcontainers base configuration

As a developer, I want to set up a shared Testcontainers configuration so all integration test classes use the same PostgreSQL container and Flyway migrations run once.

Description: Create `com.talenteo.hr.BaseIntegrationTest` (in `src/test`):

```java
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureMockMvc
@Testcontainers
public abstract class BaseIntegrationTest {

    @Container
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:15")
        .withDatabaseName("talenteo_test")
        .withUsername("test")
        .withPassword("test");

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired
    protected MockMvc mockMvc;

    @Autowired
    protected ObjectMapper objectMapper;
}
```

Acceptance Criteria:

- ☑ PostgreSQL container starts before tests
- ☑ Flyway migrations run automatically at startup
- ☑ `MockMvc` and `ObjectMapper` are available to all subclasses
- ☑ Container is shared (static) — not restarted per test class

Technical Notes:

- TD-006: Testcontainers with real PostgreSQL
- Both `EmployeeControllerIT` and `PayrollControllerIT` extend this class

US-33 — Integration test: EmployeeControllerIT

As a developer, I want to integration test all employee endpoints against a real PostgreSQL container so the full request-to-DB flow is verified.

Description: Create `com.talenteo.hr.employee.controller.EmployeeControllerIT` extending `BaseIntegrationTest`.

Test cases:

1. `POST /api/v1/employees` — valid payload → 201, response has id and status=ACTIVE
2. `POST /api/v1/employees` — missing required field → 400 with `errors` map
3. `POST /api/v1/employees` — duplicate email → 409
4. `GET /api/v1/employees/{id}` — existing id → 200 with full employee
5. `GET /api/v1/employees/{id}` — unknown id → 404
6. `GET /api/v1/employees` — no filters → 200 paginated list
7. `GET /api/v1/employees?status=ACTIVE` — only active employees returned
8. `PUT /api/v1/employees/{id}` — valid update → 200 with updated fields
9. `PUT /api/v1/employees/{id}` — email taken by another → 409
10. `DELETE /api/v1/employees/{id}` — existing → 204, employee status becomes INACTIVE
11. `DELETE /api/v1/employees/{id}` — unknown id → 404

Acceptance Criteria:

- ☑ All 11 test cases pass against real PostgreSQL
- ☑ Each test is independent (clean state or uses unique emails)
- ☑ Response body shape is asserted (not just status code)
- ☑ 400 errors include the `errors` map field

US-34 — Integration test: PayrollControllerIT

As a developer, I want to integration test all payroll endpoints against a real PostgreSQL container so the full payroll flow including edge cases is verified.

Description: Create `com.talenteo.hr.payroll.controller.PayrollControllerIT` extending `BaseIntegrationTest`.

Test cases:

1. `POST .../payroll/2026/3` — active employee → 201, DRAFT slip with correct line items and netSalary
2. `POST .../payroll/2026/3` — inactive employee → 422
3. `POST .../payroll/2026/3` — duplicate period → 409
4. `POST .../payroll/2026/3` — unknown employee → 404
5. `POST /api/v1/payroll/bulk` — all active, no existing slips → 200, all in `created`
6. `POST /api/v1/payroll/bulk` — some have existing slips → skipped entries in result

7. `POST .../payroll/2026/3/finalize` — DRAFT slip → 200, status=FINALIZED
8. `POST .../payroll/2026/3/finalize` — already finalized → 409
9. `POST .../payroll/2026/3/finalize` — slip not found → 404
10. `GET .../payroll/2026/3` — existing slip → 200 with line items
11. `GET .../payroll/2026/3` — not found → 404
12. `GET .../payroll` — existing history → 200 paginated, ordered by period descending

Acceptance Criteria:

- ☑ All 12 test cases pass against real PostgreSQL
- ☑ Line item amounts are verified for at least one happy path test
- ☑ `netSalary` is verified against expected calculation
- ☑ Bulk result `created` and `skipped` lists are asserted

Story Summary

Epic	Stories	IDs
EPIC-1: Project Setup	8	US-01 to US-08
EPIC-2: Common Infrastructure	2	US-09 to US-10
EPIC-3: Employee Management	8	US-11 to US-18
EPIC-4: Payroll Feature	10	US-19 to US-28 (excl. US-25, incl. US-25.5)
EPIC-4.5: Bulk Payroll Processing	2	US-25, US-28.5
EPIC-5: Testing	6	US-29 to US-34
Total	**35**	